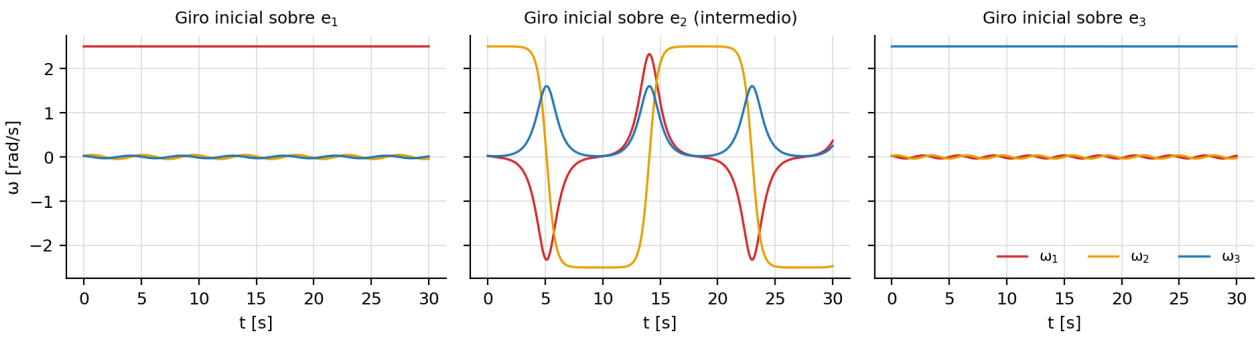
PEI 1 · Dinámica de actitud

El teorema del eje intermedio

Teoría y guía del código de PEI__1_2

Dinámica de actitud de un sólido rígido libre, efecto Dzhanibekov, integradores numéricos (Euler explícito, Euler implícito, Crank-Nicolson, Runge-Kutta 4, Leapfrog), Newton y jacobianos, regiones de estabilidad absoluta, convergencia y estructura del repositorio.



Ampliación de Matemáticas 1 · AM1GRUPO1 · PEI 1 | noviembre de 2026
Versión actualizada el 3 de noviembre de 2026 al código del commit 74d9ef2.

Índice

1. Qué contiene este documento **3**

2. El teorema del eje intermedio **3**

2.1 Ecuaciones de Euler del sólido libre . . . 3
2.2 Integrales primeras: energía y momento angular . . . 4
2.3 Equilibrios y linealización: el jacobiano . . . 4
2.4 Estabilidad no lineal: por qué los centros son de verdad estables . . . 5
2.5 Separatriz y efecto Dzhanibekov . . . 5
2.6 Actitud: de ω(t) a la orientación R(t) . . . 6

3. Estructura del repositorio **7**

3.1 Dónde está cada cosa . . . 8

4. Motor numérico: integradores **9**

4.1 El problema de Cauchy y la interfaz común . . . 9
4.2 Los cinco esquemas . . . 9
4.3 Newton-Raphson para los esquemas implícitos . . . 10
4.4 Los dos jacobianos del proyecto . . . 10

5. Estabilidad numérica: regiones de estabilidad absoluta **11**

5.1 Ecuación test y función de estabilidad . . . 11
5.2 Leapfrog: un método de dos pasos . . . 12
5.3 Cálculo en el código . . . 12
5.4 Interpretación para el sólido rígido . . . 13
5.5 Detección de divergencia en la simulación . . . 14

6. Convergencia y estimación del error **14**

6.1 Orden de convergencia: converge_sys . . . 14
6.2 Extrapolación de Richardson: richardson_error . . . 15

7. La interfaz gráfica (GUI_actitud) **15**

7.1 Distribución del panel . . . 15
7.2 Parámetros . . . 16
7.3 Bucle de simulación . . . 16
7.4 Catálogo de cuerpos y tensor de inercia . . . 16

8. Observaciones sobre el código **17**

8.1 Región de estabilidad de Leapfrog: corregida . . . 17
8.2 El jacobiano analítico no se usa en Newton . . . 18
8.3 Dependencia cruzada entre paquetes: resuelta . . . 18
8.4 El ρ de Leapfrog se calcula pero no se usa . . . 18
8.5 Los pasos de converge_sys no son potencias exactas de 2 . . . 18

9. Resumen **18**

1. Qué contiene este documento

Este documento explica, por un lado, la física y las matemáticas del **teorema del eje intermedio** (también llamado teorema de la raqueta de tenis o efecto Dzhanibekov) y, por otro, cómo está implementado en el repositorio `PEI__1_2`: en qué archivo y función está cada ecuación, cada integrador, cada jacobiano y cada cálculo de estabilidad.

Las referencias al código usan el formato `carpeta/archivo.py:línea`, relativo a la carpeta `PEI__1_2/`. Todas las figuras de resultados se han generado llamando directamente a las funciones del repositorio, con los valores iniciales de los deslizadores del panel salvo que se indique otra cosa: I = (0.21, 0.42, 0.58), Ω = 2.5 rad/s, ε = 0.01, Δt = 0.01 s.

Para ejecutar la aplicación basta con `python main.py` desde la carpeta `PEI__1_2/`; las únicas dependencias son **numpy** y **matplotlib**.

> **Cambios respecto a la versión del 1 de noviembre**
> - El cálculo de las regiones de estabilidad se ha sustituido por el método numérico del profesor (commit 6fb6fea): ahora se aplica `paso_Cauchy` a la ecuación test en cada punto de la malla. §5.3 está reescrita y la figura 4 se ha regenerado con el código nuevo.
> - La región de Leapfrog ya se dibuja bien (antes era la observación 8.1). §8.1 explica la corrección y la figura 7 compara el antes y el ahora.
> - `obtener_region_estabilidad` ya no existe; la sustituye `region_por_esquema`.
> - Las referencias a `GUI_actitud/panel.py` posteriores a la línea 880 se han desplazado 30 líneas (por ejemplo, `_actualizar_analisis` pasa de la 905 a la 935).
> - `estabilidad` (las R(z) en forma cerrada) se ha movido de `space_physics/stability.py`, que ya no existe, a `numerical_engine/stability.py`. El motor numérico ya no depende del paquete de física (antigua observación 8.3, resuelta); la figura 3 está actualizada.
> - `estabilidad("LF", z)` devuelve ahora el máximo de los módulos de las dos raíces, y `stability_report` ya no necesita un caso especial para Leapfrog.
> - Los `.pyc` ya no están versionados y hay `.gitignore` (antigua observación 8.4, resuelta). §8 incluye dos observaciones nuevas (8.4 y 8.5).

2. El teorema del eje intermedio

Todo sólido rígido tiene tres **ejes principales de inercia** e_1, e_2, e_3, con momentos principales $I_1 < I_2 < I_3$. El teorema afirma que, en ausencia de momentos externos, la rotación uniforme alrededor del eje de inercia **mínima** o **máxima** es estable, mientras que la rotación alrededor del eje **intermedio** es inestable: cualquier perturbación, por pequeña que sea, hace que el cuerpo acabe dando media vuelta ("volteo") y repita el proceso periódicamente. Es lo que se observa al lanzar una raqueta, un libro o un móvil girando sobre su eje intermedio, y lo que el cosmonauta Vladímir Dzhanibekov filmó en 1985 con una tuerca en forma de T a bordo de la Salyut 7.

2.1 Ecuaciones de Euler del sólido libre

En ejes cuerpo (ligados al sólido y alineados con los ejes principales), la conservación del momento angular $\mathbf{L} = \mathbf{I}\omega$ sin momentos externos se escribe como

$$\mathbf{I}\,\dot{\vec{\omega}} + \vec{\omega} \times (\mathbf{I}\,\vec{\omega}) = \vec{0}$$

que, componente a componente y despejando las derivadas, da el sistema autónomo no lineal $d\omega/dt = F(\omega)$:

$$\dot{\omega}_1 = \frac{I_2 - I_3}{I_1}\,\omega_2\,\omega_3, \qquad \dot{\omega}_2 = \frac{I_3 - I_1}{I_2}\,\omega_3\,\omega_1, \qquad \dot{\omega}_3 = \frac{I_1 - I_2}{I_3}\,\omega_1\,\omega_2$$

En el código: `ecuaciones_rotacion(W, I1, I2, I3)` en `space_physics/dynamics.py:3`. El panel la envuelve como función de Cauchy F(W, t) en `GUI_actitud/panel.py:650` (la t se ignora: el sistema es autónomo).

Conviene llamar $a_1 = (I_2-I_3)/I_1 < 0$, $a_2 = (I_3-I_1)/I_2 > 0$ y $a_3 = (I_1-I_2)/I_3 < 0$; los signos son consecuencia directa de $I_1 < I_2 < I_3$ y son los que deciden la estabilidad.

2.2 Integrales primeras: energía y momento angular

El sistema conserva dos cantidades cuadráticas: la energía cinética de rotación T y el módulo al cuadrado del momento angular L²:

$$2T = I_1\omega_1^2 + I_2\omega_2^2 + I_3\omega_3^2, \qquad L^2 = I_1^2\omega_1^2 + I_2^2\omega_2^2 + I_3^2\omega_3^2$$

Geométricamente, ω vive a la vez sobre el **elipsoide de energía** (semiejes $\sqrt{(2T/I_k)}$) y sobre el **elipsoide de momento** (semiejes L/I_k). Su intersección es una curva cerrada llamada **polhodia**, que es la trayectoria de ω en el espacio de fases. Que las dos superficies se corten exige

$$2I_1T \le L^2 \le 2I_3T$$

En el código: `invariantes(W, I1, I2, I3)` devuelve (T, L²) en `space_physics/invariants.py:3` y `historial_invariantes(U, ...)` hace lo mismo vectorizado sobre toda la trayectoria (`space_physics/invariants.py:8`). El panel usa estas funciones para la vista *Deriva de invariantes* (`GUI_actitud/panel.py:1089`) y comprueba la desigualdad anterior en la vista *Análisis espectral* (`GUI_actitud/panel.py:955`). Los semiejes de los dos elipsoides se calculan en `_preparar_fase` (`GUI_actitud/panel.py:817`) y las cotas de $|\omega_k|$ sobre la polhodia en `_cotas_polhodia` (`GUI_actitud/geometry.py:36`).

2.3 Equilibrios y linealización: el jacobiano

Las rotaciones uniformes $\omega = \Omega\, e_k$ alrededor de cualquier eje principal son puntos de equilibrio, porque en cada ecuación aparece el producto de las otras dos componentes, que es nulo. Para estudiar su estabilidad se linealiza F alrededor del equilibrio con la matriz jacobiana $J = \partial F/\partial\omega$:

$$J(\omega) = \begin{bmatrix} 0 & a_1\omega_3 & a_1\omega_2 \\ a_2\omega_3 & 0 & a_2\omega_1 \\ a_3\omega_2 & a_3\omega_1 & 0 \end{bmatrix}$$

En el código: el jacobiano analítico es `jacobiano(W, I1, I2, I3)` en `space_physics/dynamics.py:12`. La función `estabilidad_ejes(I1, I2, I3, omega)` (`space_physics/equilibria.py:5`) construye los tres equilibrios $\Omega\, e_k$, evalúa en ellos el jacobiano y calcula sus autovalores con `np.linalg.eigvals`.

Sustituyendo cada equilibrio, la matriz tiene una fila nula (la componente que gira no varía a primer orden) y un bloque 2×2 antidiagonal cuyos autovalores son $\pm\Omega\sqrt{}$(producto de las dos a):

$$\omega = \Omega e_1 : \;\; \lambda = 0, \;\; \pm\, i\Omega\sqrt{\frac{(I_3 - I_1)(I_2 - I_1)}{I_2 I_3}} \qquad \text{(centro)}$$

$$\omega = \Omega e_2 : \;\; \lambda = 0, \;\; \pm\, \Omega\sqrt{\frac{(I_3 - I_2)(I_2 - I_1)}{I_1 I_3}} \qquad \text{(silla)}$$

$$\omega = \Omega e_3 : \;\; \lambda = 0, \;\; \pm\, i\Omega\sqrt{\frac{(I_3 - I_1)(I_3 - I_2)}{I_1 I_2}} \qquad \text{(centro)}$$

En los ejes extremos el producto de coeficientes es negativo y los autovalores son imaginarios puros (oscilación de la perturbación); en el eje intermedio es positivo y aparece un autovalor real positivo σ, de modo que la perturbación crece como $e^{\sigma t}$. El autovalor nulo corresponde a la dirección del propio equilibrio (cambiar Ω da otro equilibrio). Con los parámetros por defecto:

Eje	Autovalores (código)	Tipo	Veredicto en el panel
e_1 (I mínimo)	0, ±1.4119 i	centro	centro · estable
e_2 (I intermedio)	0, ±1.3131	silla	silla · inestable
e_3 (I máximo)	0, ±2.0482 i	centro	centro · estable

El veredicto se muestra en la vista *Análisis espectral*; lo calcula `_actualizar_analisis` (`GUI_actitud/panel.py:935`), que clasifica según la parte real/imaginaria del autovalor dominante.

2.4 Estabilidad no lineal: por qué los centros son de verdad estables

La linealización solo prueba que e_2 es inestable (un autovalor con parte real positiva basta). Para e_1 y e_3 los autovalores imaginarios dejan el caso lineal indeciso; la estabilidad se demuestra con las integrales primeras, que actúan como funciones de Lyapunov:

$$L^2 - 2I_1 T = I_2(I_2 - I_1)\,\omega_2^2 + I_3(I_3 - I_1)\,\omega_3^2 \;\geq\; 0$$

$$2I_3 T - L^2 = I_1(I_3 - I_1)\,\omega_1^2 + I_2(I_3 - I_2)\,\omega_2^2 \;\geq\; 0$$

La primera es definida positiva en (ω_2, ω_3) y se conserva: si empiezan pequeñas, permanecen pequeñas, luego Ω e_1 es estable. Igual con la segunda para e_3. Para el eje intermedio, en cambio,

$$L^2 - 2I_2 T = -\,I_1(I_2 - I_1)\,\omega_1^2 + I_3(I_3 - I_2)\,\omega_3^2$$

es **indefinida**: no confina la trayectoria y ω puede alejarse de e_2 sin violar ninguna ley de conservación.

2.5 Separatriz y efecto Dzhanibekov

Las órbitas que pasan exactamente por los equilibrios ±Ω e_2 cumplen $L^2 = 2I_2 T$. Anulando la expresión anterior se obtienen dos planos que contienen al eje 2:

$$\omega_3 = \pm\, k\,\omega_1, \qquad k = \sqrt{\frac{I_1\,(I_2 - I_1)}{I_3\,(I_3 - I_2)}}$$

La intersección de estos planos con el elipsoide de energía son dos elipses: la **separatriz**, formada por órbitas heteroclínicas que unen +Ω e_2 con −Ω e_2. Divide el elipsoide en cuatro regiones: dos con polhodias alrededor de ±e_1 y dos alrededor de ±e_3. Una condición inicial

cercana a e_2 recorre una órbita próxima a la separatriz: pasa mucho tiempo cerca de $+e_2$, cruza rápidamente a $-e_2$ (el volteo) y vuelve. El tiempo entre volteos crece como $\ln(1/\varepsilon)/\sigma$ al acercarse a la separatriz.

En el código: `separatriz(I1, I2, I3, energia)` en `space_physics/equilibria.py:21` calcula `pendiente` = k y parametriza la elipse como $\omega_1 = A\cos\varphi$, $\omega_2 = B\sin\varphi$, $\omega_3 = k\,\omega_1$, con A y B fijados por la energía. El panel dibuja las dos ramas (k y −k) en el mapa de fases (`GUI_actitud/panel.py:830`).

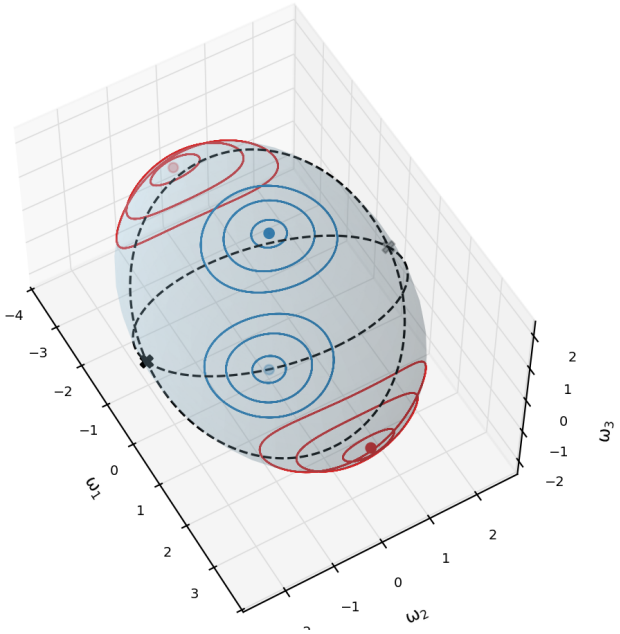


Figura 1. Polhodias sobre el elipsoide de energía (integradas con RK4 del repositorio): en rojo alrededor de $\pm e_1$, en azul alrededor de $\pm e_3$. Línea discontinua: separatriz generada por separatriz(). Las cruces marcan las sillas $\pm\Omega\, e_2$.

La figura de la portada (repetida aquí por claridad) muestra ω(t) para los tres ejes con ε = 0.01: en e_1 y e_3 las componentes transversales oscilan con amplitud ~ εΩ, mientras que en e_2 ω_2 cambia de signo periódicamente: cada cambio es un volteo del cuerpo.

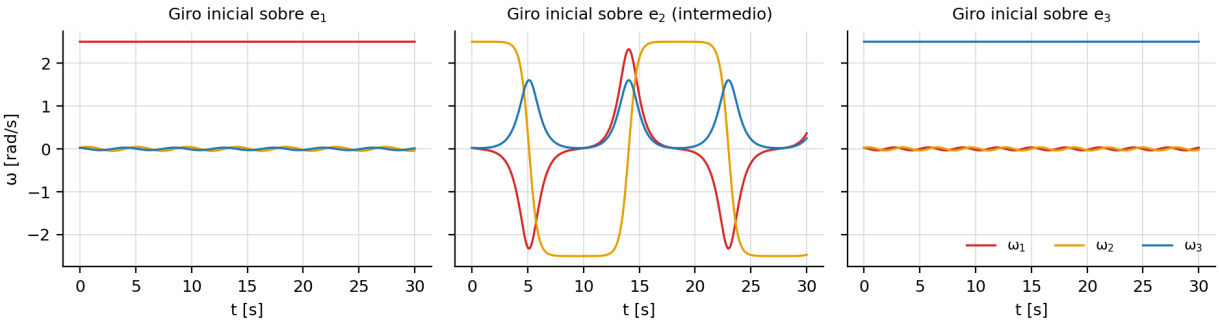


Figura 2. ω(t) con RK4, Δt = 0.01 s, partiendo de $\Omega\, e_k$ + εΩ (1,1,1). Centro: efecto Dzhanibekov.

2.6 Actitud: de ω(t) a la orientación R(t)

Las ecuaciones de Euler solo dan ω en ejes cuerpo. Para dibujar el sólido hace falta su orientación R(t) ∈ SO(3) (matriz que pasa de ejes cuerpo a ejes inerciales), que cumple $\dot{R} = R\,\hat{\Omega}$, con $\hat{\Omega}$ la matriz antisimétrica del producto vectorial. El código la integra con la fórmula de Rodrigues usando la velocidad media del paso:

$$R_{n+1} = R_n(\mathrm{Id} + \sin\theta\, S + (1-\cos\theta)\, S^2), \qquad \theta = |\bar{\omega}|\,\Delta t, \quad \bar{\omega} = \frac{\omega_n + \omega_{n+1}}{2}$$

donde S es la matriz antisimétrica del vector unitario ω̄/|ω̄|. Al ser la exponencial exacta de una rotación, R se mantiene ortogonal sin necesidad de reortonormalizar. En el código: `paso_actitud` (`space_physics/attitude.py:3`) hace un paso y `actitud` (`space_physics/attitude.py:14`) recorre una trayectoria completa. Para la animación, `_alinear_con_z` (`GUI_actitud/geometry.py:4`) calcula una rotación fija P que lleva el momento angular inicial $\mathbf{I}\omega_0$ al eje vertical; como **L** es constante en ejes inerciales, en pantalla **L** siempre apunta hacia arriba y el cuerpo se dibuja con P·R (`GUI_actitud/panel.py:988`).

Los volteos se cuentan en `_avanzar` (`GUI_actitud/panel.py:700`): se sigue la punta del eje de giro inicial y se suma un volteo cada vez que su componente vertical cambia de signo con un margen de 0.5.

3. Estructura del repositorio

El proyecto se divide en tres paquetes con responsabilidades separadas, más el punto de entrada `main.py` (que crea `PanelActitud`, maximiza la ventana y llama a `plt.show()`).

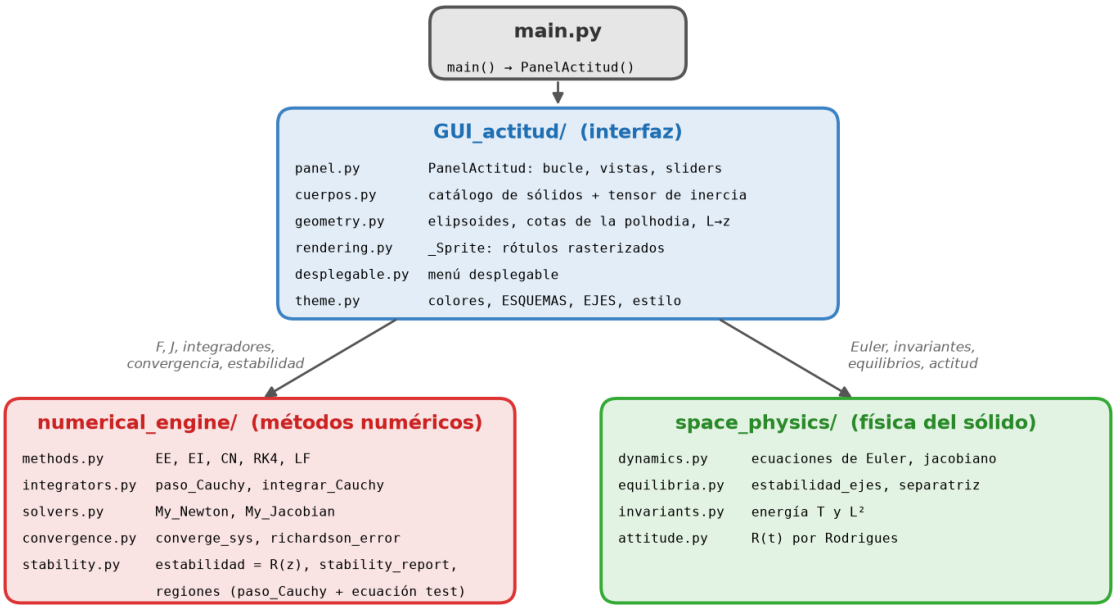


Figura 3. Paquetes y dependencias. La GUI usa ambos paquetes; numerical_engine y space_physics no dependen el uno del otro.

```
PEI__1_2/
├─ main.py                  punto de entrada
├─ numerical_engine/        métodos numéricos genéricos (no saben nada de sólidos)
│   ├─ methods.py           un paso de EE, EI, CN, RK4, LF
│   ├─ integrators.py       paso_Cauchy (despachador) e integrar_Cauchy (bucle)
│   ├─ solvers.py           Newton-Raphson y jacobiano por diferencias finitas
│   ├─ convergence.py       orden de convergencia y extrapolación de Richardson
│   └─ stability.py         R(z) de cada esquema, |R(Δt·λ)| y regiones de
│                           estabilidad (paso_Cauchy aplicado a la ecuación test)
├─ space_physics/           modelo físico del sólido rígido libre
│   ├─ dynamics.py          ecuaciones de Euler y su jacobiano analítico
│   ├─ equilibria.py        equilibrios, autovalores y separatriz
│   ├─ invariants.py        energía T y momento angular L²
│   └─ attitude.py          orientación R(t) con Rodrigues
└─ GUI_actitud/             interfaz interactiva en matplotlib
    ├─ panel.py             PanelActitud: ventana, bucle de simulación, vistas
```

```
├─ cuerpos.py            catálogo de 6 sólidos y su tensor de inercia
├─ geometry.py           elipsoides, cotas de la polhodia, alineación L→z
├─ rendering.py          _Sprite: texto rasterizado para la animación
├─ desplegable.py        menú desplegable (matplotlib no trae uno)
└─ theme.py              colores, nombres de esquemas y ejes, estilo
```

3.1 Dónde está cada cosa

Concepto	Función	Ubicación
Ecuaciones de Euler F(ω)	`ecuaciones_rotacion`	`space_physics/dynamics.py:3`
Jacobiano analítico ∂F/∂ω	`jacobiano`	`space_physics/dynamics.py:12`
Equilibrios, jacobianos y autovalores en e_1, e_2, e_3	`estabilidad_ejes`	`space_physics/equilibria.py:5`
Separatriz (pendiente k)	`separatriz`	`space_physics/equilibria.py:21`
Energía T y L^2	`invariantes`, `historial_invariantes`	`space_physics/invariants.py:3, 8`
Orientación R(t) (Rodrigues)	`paso_actitud`, `actitud`	`space_physics/attitude.py:3, 14`
Euler explícito	`EE`	`numerical_engine/methods.py:5`
Euler implícito	`EI`	`numerical_engine/methods.py:8`
Crank-Nicolson	`CN`	`numerical_engine/methods.py:15`
Runge-Kutta 4	`RK4`	`numerical_engine/methods.py:28`
Leapfrog	`LF`	`numerical_engine/methods.py:35`
Selección del esquema en cada paso	`paso_Cauchy`	`numerical_engine/integrators.py:7`
Integración completa (arranque LF con RK4)	`integrar_Cauchy`	`numerical_engine/integrators.py:29`
Newton para esquemas implícitos	`My_Newton`	`numerical_engine/solvers.py:6`
Jacobiano numérico (dif. centradas)	`My_Jacobian`	`numerical_engine/solvers.py:27`
Orden de convergencia (ajuste log-log)	`converge_sys`	`numerical_engine/convergence.py:7`
Error y orden por Richardson	`richardson_error`	`numerical_engine/convergence.py:26`
Funciones de estabilidad R(z)	`estabilidad`	`numerical_engine/stability.py:3`
max |R(Δt·λ)| para unos autovalores	`stability_report`	`numerical_engine/stability.py:18`
Malla ρ(z) de los cinco esquemas	`obtener_todas_regiones_estabilidad`	`numerical_engine/stability.py:23`
ρ(z) de un esquema aplicando un paso a la ecuación test	`region_por_esquema`	`numerical_engine/stability.py:36`
Condición inicial ω_0, F y J del panel	`PanelActitud.reiniciar`	`GUI_actitud/panel.py:639`
Un paso de simulación + detección de divergencia y volteos	`PanelActitud._avanzar`	`GUI_actitud/panel.py:676`
Bucle temporal (timer)	`PanelActitud._tick`	`GUI_actitud/panel.py:705`
Gráfico de regiones de estabilidad	`_actualizar_estabilidad_plot`	`GUI_actitud/panel.py:860`
Gráfico de convergencia + Richardson	`mostrar_convergencia`, `calcular_richardson`	`GUI_actitud/panel.py:762, 722`
Texto de análisis espectral	`_actualizar_analisis`	`GUI_actitud/panel.py:935`

Concepto	Función	Ubicación
Mapa de fases (elipsoides, separatriz)	`_preparar_fase`	`GUI_actitud/panel.py:817`
Tensor de inercia de cada sólido	`Cuerpo.__init__`	`GUI_actitud/cuerpos.py:207`
Nombres de esquemas y ejes de la GUI	`ESQUEMAS`, `EJES`	`GUI_actitud/theme.py:35, 43`

4. Motor numérico: integradores

4.1 El problema de Cauchy y la interfaz común

Todos los esquemas resuelven un problema de Cauchy dU/dt = F(U, t), $U(t_0) = U_0$ con paso constante $\Delta t = (t_f - t_0)/N$. El motor es genérico: recibe F (y opcionalmente su jacobiano J) como funciones F(U, t) y J(U, t), así que no depende del sólido rígido. Hay dos niveles:

- `paso_Cauchy(esquema, U, U_prev, At, t, F, J)` (`numerical_engine/integrators.py:7`): despachador que, según la cadena "EE", "EI", "CN", "RK4" o "LF", llama al método correspondiente y devuelve U^{n+1}. Si recibe un nombre desconocido usa EE. Es lo que llama el panel en cada paso de la animación y, desde esta versión, también el cálculo de las regiones de estabilidad (§5.3).
- `integrar_Cauchy(esquema, U0, N_t, t0, tf, F, J)` (`numerical_engine/integrators.py:29`): reserva la matriz U de tamaño (N+1)×dim y aplica `paso_Cauchy` N veces. Para Leapfrog, que es de dos pasos, calcula U^1 con un paso de RK4 y después aplica la recurrencia. Lo usan los estudios de convergencia.

El panel no llama a `integrar_Cauchy` durante la animación: avanza paso a paso en `_avanzar` (`GUI_actitud/panel.py:676`) para poder dibujar en tiempo real, y reproduce el mismo arranque de LF con RK4 cuando n = 0.

4.2 Los cinco esquemas

Todos están en `numerical_engine/methods.py`; cada función recibe el estado actual y devuelve el siguiente.

Euler explícito (EE) · línea 5

$$U^{n+1} = U^n + \Delta t\, F(U^n, t_n)$$

Explícito, de un paso, orden 1. Una evaluación de F por paso.

Euler implícito (EI) · línea 8

$$U^{n+1} = U^n + \Delta t\, F(U^{n+1}, t_{n+1})$$

Implícito, orden 1. La incógnita aparece dentro de F, así que cada paso exige resolver un sistema no lineal; `EI` delega directamente en `My_Newton`.

Crank-Nicolson (CN) · línea 15

$$U^{n+1} = U^n + \frac{\Delta t}{2}\left[F(U^n, t_n) + F(U^{n+1}, t_{n+1})\right]$$

Implícito, orden 2 (regla del trapecio). El código reutiliza el mismo Newton que EI con un truco: define G(X) = (F(U, t) + F(X, time)) / 2 y su jacobiano JG(X) = J(X) / 2; entonces la ecuación de Euler implícito aplicada a G, Y = U + Δt G(Y), es exactamente Crank-Nicolson.

Runge-Kutta 4 (RK4) · línea 28

$$k_1 = F(U^n, t_n), \quad k_2 = F(U^n + \tfrac{\Delta t}{2} k_1,\ t_n + \tfrac{\Delta t}{2}), \quad k_3 = F(U^n + \tfrac{\Delta t}{2} k_2,\ t_n + \tfrac{\Delta t}{2})$$

$$k_4 = F(U^n + \Delta t\, k_3,\ t_n + \Delta t), \qquad U^{n+1} = U^n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

Explícito, orden 4, cuatro evaluaciones de F por paso. Es el esquema por defecto del panel.

Leapfrog (LF) · línea 35

$$U^{n+1} = U^{n-1} + 2\,\Delta t\, F(U^n, t_n)$$

Explícito, de dos pasos, orden 2 (diferencias centradas). Necesita U^{n-1}, que se pasa como `U_prev`, y un arranque para U^1 (se usa RK4).

Esquema	Tipo	Pasos	Orden teórico	Evals. de F / paso	Necesita J
EE	explícito	1	1	1	no
EI	implícito	1	1	1 + Newton	sí (Newton)
CN	implícito	1	2	2 + Newton	sí (Newton)
RK4	explícito	1	4	4	no
LF	explícito	2	2	1	no

4.3 Newton-Raphson para los esquemas implícitos

`My_Newton(U, At, t, F, J, tol=1e-10, max_iter=50)` (`numerical_engine/solvers.py:6`) resuelve la ecuación de Euler implícito para Y = U^{n+1}:

$$\Phi(Y) = Y - U^n - \Delta t\, F(Y, t_{n+1}) = 0$$

$$\left(\mathrm{Id} - \Delta t J(Y^{(k)})\right)\delta = \Phi(Y^{(k)}), \qquad Y^{(k+1)} = Y^{(k)} - \delta$$

- **Predictor:** la iteración arranca en $Y^{(0)}$ = U + Δt F(U, t), es decir, un paso de Euler explícito.
- **Criterio de parada:** $\|\Phi(Y)\| \le 10^{-10}$. Si no se alcanza en 50 iteraciones lanza `RuntimeError`; el panel lo captura y muestra «EL ESQUEMA DIVERGE».
- **Matriz del sistema:** Id − Δt J, que es el jacobiano de Φ. Si no se pasa J, se aproxima con `My_Jacobian`.

4.4 Los dos jacobianos del proyecto

En el repositorio hay dos jacobianos distintos con papeles distintos, y conviene no confundirlos:

	Jacobiano analítico	Jacobiano numérico
Función	`jacobiano(W, I1, I2, I3)`	`My_Jacobian(F, U, t)`
Archivo	`space_physics/dynamics.py:12`	`numerical_engine/solvers.py:27`
Cómo se calcula	Derivadas exactas de las ecuaciones de Euler (matriz de §2.3).	Diferencias finitas centradas columna a columna: $J_{:,i} \approx [F(U+h\,e_i) - F(U-h\,e_i)]/(2h)$, con $h = \varepsilon_{máq}^{1/3} \cdot \max(1, \lvert U_i \rvert) \approx 6 \cdot 10^{-6}$.
Para qué se usa	Análisis de estabilidad lineal de los equilibrios: autovalores λ de cada eje (`estabilidad_ejes`), texto del análisis espectral y puntos Δt·λ en el gráfico de regiones.	Matriz de Newton en EI y CN. El panel lo fija en `GUI_actitud/panel.py:651`: `self.J = lambda W, t: My_Jacobian(self.F, W, t)`.
Error	Exacto.	$O(h^2)$ de truncamiento + $O(\varepsilon/h)$ de redondeo; el h elegido equilibra ambos ($\sim 10^{-10}$ relativo).

El paso $h \sim \varepsilon^{1/3}$ no es arbitrario: para una diferencia centrada el error total es $\approx C_1 h^2 + C_2 \varepsilon/h$, que se minimiza con $h \propto \varepsilon^{1/3}$.

5. Estabilidad numérica: regiones de estabilidad absoluta

5.1 Ecuación test y función de estabilidad

La estabilidad de un esquema se estudia aplicándolo a la ecuación lineal escalar $dy/dt = \lambda y$, $\lambda \in \mathbb{C}$. Todos los esquemas de un paso dan entonces una recurrencia $U^{n+1} = R(z)\,U^n$ con $z = \lambda \Delta t$. La **región de estabilidad absoluta** es el conjunto de z con $|R(z)| \le 1$: si z cae dentro, la solución numérica no crece.

$$\dot{y} = \lambda y \implies U^{n+1} = R(z)\,U^n, \qquad z = \lambda\,\Delta t$$

La conexión con el sólido rígido es la linealización de §2.3: cerca de un equilibrio, la perturbación evoluciona como $d\delta\omega/dt \approx J\,\delta\omega$, que se descompone en modos $e^{\lambda t}$ con los autovalores λ del jacobiano. Por eso el panel coloca los puntos Δt·λ del eje activo sobre el diagrama de regiones.

Esquema	R(z)	Región $\lvert R(z) \rvert \le 1$	Sobre el eje imaginario z = iy
EE	$1 + z$	Disco de centro −1 y radio 1	$\lvert R \rvert = \sqrt{1+y^2} > 1$: amplifica
EI	$1 / (1 - z)$	Exterior del disco de centro +1 y radio 1 (A-estable)	$\lvert R \rvert = 1/\sqrt{1+y^2} < 1$: disipa
CN	$(1 + z/2) / (1 - z/2)$	Semiplano Re $z \le 0$ (A-estable)	$\lvert R \rvert = 1$ exacto: conserva
RK4	$1 + z + z^2/2 + z^3/6 + z^4/24$	Región acotada; corta el eje real en −2.785 y el imaginario en $\lvert y \rvert \le 2\sqrt{2}$	$\lvert R \rvert^2 = 1 - y^6/72 + y^8/576$: disipa muy poco si $\lvert y \rvert < 2\sqrt{2}$
LF	$r = z \pm \sqrt{1+z^2}$ (dos raíces)	Solo el segmento imaginario [−i, i]	$\lvert r \rvert = 1$ si $\lvert y \rvert \le 1$: conserva; si no, explota

En el código: `estabilidad(esquema, z)` en `numerical_engine/stability.py:3` implementa estas R(z) en forma cerrada y vectorizada; para LF devuelve $\max(\lvert r_1 \rvert, \lvert r_2 \rvert)$ (§5.2). La usa `stability_report`; las regiones del gráfico se calculan aplicando los propios esquemas (§5.3).

5.2 Leapfrog: un método de dos pasos

Leapfrog no tiene una única R(z). Aplicado a la ecuación test da $U^{n+1} - 2z\,U^{n} - U^{n-1} = 0$, cuyo polinomio característico es

$$r^2 - 2zr - 1 = 0, \qquad r_{1,2} = z \pm \sqrt{1+z^2}, \qquad r_1 r_2 = -1$$

Como el producto de las raíces tiene módulo 1, las dos solo pueden ser ≤ 1 en módulo si ambas valen exactamente 1, lo que ocurre si y solo si $z \in [-i, i]$. Leapfrog es, por tanto, estable solo para problemas oscilatorios puros con $|\lambda\Delta t| \le 1$, y en ellos no introduce ni amplificación ni disipación (es simétrico en el tiempo). Por eso `estabilidad("LF", z)` (`numerical_engine/stability.py:13`) devuelve el máximo de los módulos de las dos raíces. `region_por_esquema` llega al mismo polinomio por otra vía (§5.3).

5.3 Cálculo en el código

- `stability_report(esquema, eigenvalues, At)` (`numerical_engine/stability.py:18`): calcula $z = \Delta t \cdot \lambda$ para cada autovalor y devuelve (estable, max|R(z)|) usando `estabilidad`. Lo usa el texto «Estabilidad numérica del esquema» de la vista espectral (`GUI_actitud/panel.py:965`).
- `obtener_todas_regiones_estabilidad(esquemas, resolucion=85)` (`numerical_engine/stability.py:23`): llama a `region_por_esquema` para cada esquema y devuelve la malla (X, Y) y un diccionario esquema → ρ(z). El panel la llama una sola vez y la cachea en `self._regiones` (`GUI_actitud/panel.py:877`).
- `region_por_esquema(esquema, resolucion=85)` (`numerical_engine/stability.py:36`): recorre una malla 85×85 de z en $[-4, 2]\times[-3, 3]$ y, en lugar de evaluar una fórmula cerrada de R(z), **aplica el propio esquema**: define el problema test F(U, t) = zU con jacobiano J = z y da **un paso** de `paso_Cauchy` con Δt = 1 desde U = 1. Como $U^1 = R(z)\cdot 1$, el resultado es directamente R(z), y ρ = |R(z)|.

La ventaja de este método es que comprueba la **implementación** y no una fórmula escrita a mano: EI y CN pasan por el mismo `My_Newton` que se usa para integrar el sólido. Los puntos donde el paso falla (Newton no converge o la matriz es singular, por ejemplo z = 1 en EI o z = 2 en CN) quedan como NaN y el panel los enmascara con `np.ma.masked_invalid` (`GUI_actitud/panel.py:903`).

Leapfrog necesita dos estados, así que la función reconstruye su **matriz de amplificación**. Da un paso con $(U, U_{prev}) = (1, 0)$, que devuelve a = 2z, y otro con (0, 1), que devuelve b = 1:

$$U^{n+1} = a\,U^{n} + b\,U^{n-1}, \quad a = 2z,\ b = 1 \quad \Longrightarrow \quad \begin{pmatrix} U^{n+1} \\ U^{n} \end{pmatrix} = M \begin{pmatrix} U^{n} \\ U^{n-1} \end{pmatrix}, \quad M = \begin{bmatrix} 2z & 1 \\ 1 & 0 \end{bmatrix}, \quad \rho = \max|\mathrm{eig}(M)|$$

El polinomio característico de M es $r^2 - 2zr - 1 = 0$, el mismo de §5.2, de modo que $\rho = \max(|r_1|, |r_2|)$ y solo vale ≤ 1 en el segmento $[-i, i]$. Como ese conjunto tiene área nula, un contorno no lo puede dibujar: en la malla solo la columna Re z = 0 da ρ ≤ 1. Por eso `_actualizar_estabilidad_plot` (`GUI_actitud/panel.py:885`) dibuja LF directamente como el segmento $[-i, i]$; para los demás esquemas rellena ρ ≤ 1 y traza el contorno ρ = 1, y encima coloca el punto $\Delta t \cdot \lambda$ del eje activo.

Coste. Son $85^2 \times 5 \approx 36\,000$ pasos individuales (con Newton en EI y CN). En el ordenador de prueba tarda ≈ 1.2 s la primera vez que se abre la vista; después se usa la caché. La versión anterior evaluaba R(z) vectorizada en una malla 200×200, así que los contornos son ahora algo más gruesos, pero coinciden con los teóricos.

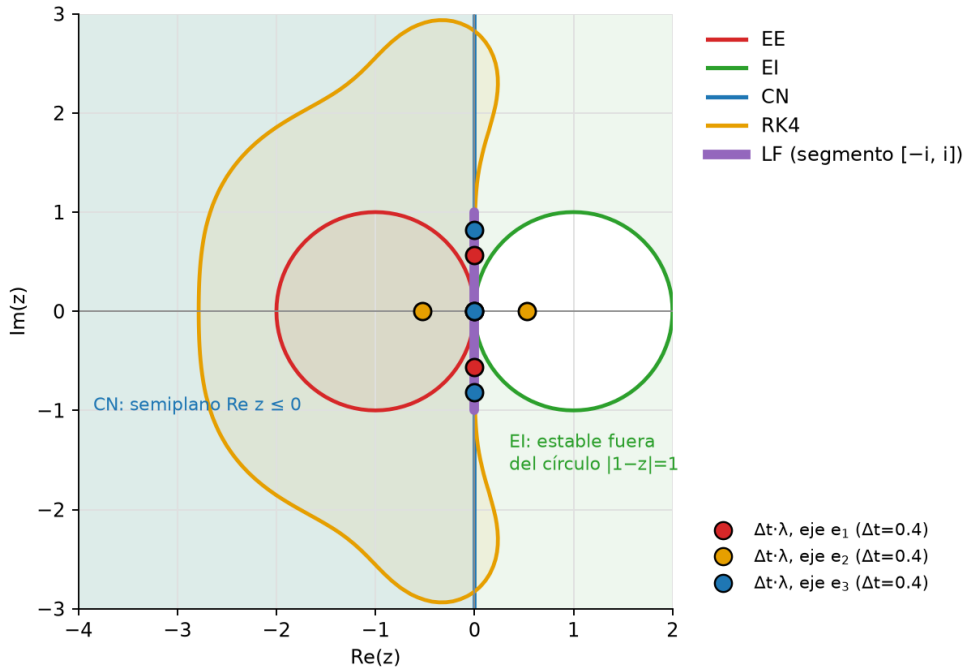


Figura 4. Regiones de estabilidad absoluta calculadas con obtener_todas_regiones_estabilidad() (contorno ρ = 1, malla 85×85 como en el panel; LF dibujado como segmento) y puntos Δt·λ de los tres ejes con un Δt = 0.4 exagerado para que se vean separados del origen. e_1 y e_3 caen sobre el eje imaginario; e_2 tiene un z real positivo, fuera de todas las regiones.

5.4 Interpretación para el sólido rígido

Ejes estables e_1, e_3. Los autovalores son imaginarios puros, así que $z = \pm i\omega\Delta t$ está sobre el eje imaginario, que es exactamente la frontera de las regiones de EE y EI. Por eso Euler explícito hace crecer la energía y Euler implícito la disipa aunque el paso sea pequeño, mientras que CN y LF la conservan y RK4 casi.

Eje inestable e_2. El autovalor $\sigma > 0$ da z real positivo, fuera de toda región: $|R| > 1$ en todos los esquemas. Aquí no es un defecto numérico, sino la inestabilidad física que el esquema debe reproducir; por eso el panel la etiqueta «crece (modo inestable)».

Eje	EE	EI	CN	RK4	LF
e_1	1.0000996718	0.9999003382	1.0000000000	1.0000000000	1.0000000000
e_2	1.0131306433	1.0133053511	1.0132174199	1.0132172287	1.0132168465
e_3	1.0002097286	0.9997903154	1.0000000000	1.0000000000	1.0000000000

Tabla: max $|R(\Delta t\cdot\lambda)|$ devuelto por `stability_report` con $\Delta t = 0.01$ s y los parámetros por defecto.

El efecto por paso es minúsculo, pero se acumula: tras N pasos el factor es $|R|^N$. La figura 5 lo confirma integrando 30 s sobre e_1.

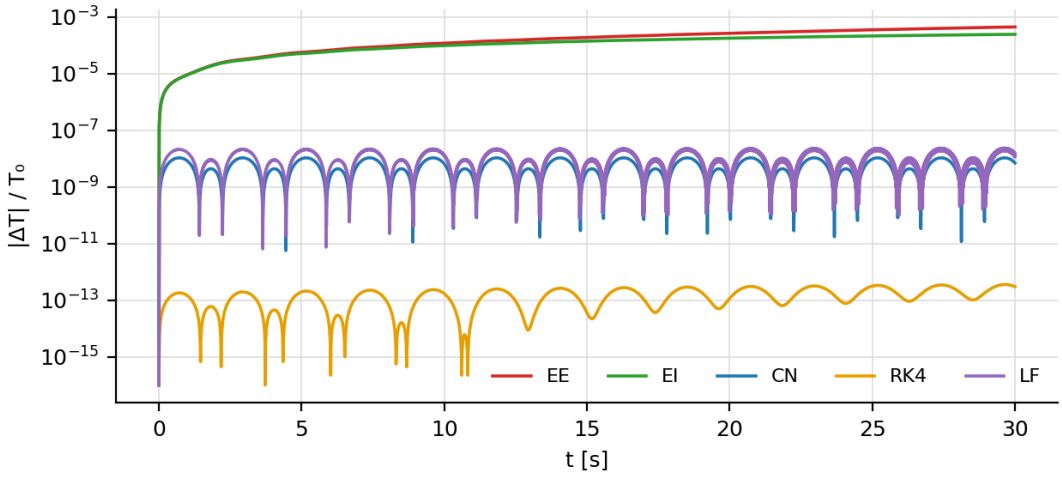


Figura 5. Deriva relativa de la energía, $|T(t)/T_0 - 1|$, al girar sobre e_1 (30 s, Δt = 0.01). Valores finales: EE 4.4e-04, EI 2.4e-04, CN 6.9e-09, RK4 3.0e-13, LF 1.1e-08.

Las curvas de EE y EI coinciden en magnitud pero tienen significado opuesto (EE gana energía, EI la pierde). CN y LF oscilan en torno a 10^{-8} sin tendencia: conservan en promedio. RK4 se queda en 10^{-13}, cerca del redondeo, porque su error por paso es $O(\Delta t^5)$.

5.5 Detección de divergencia en la simulación

Cuando Δt es demasiado grande (por ejemplo LF con $|\lambda|\Delta t > 1$ o EE durante mucho tiempo), la solución numérica explota. `_avanzar` (`GUI_actitud/panel.py:686`) detiene la simulación y muestra «EL ESQUEMA DIVERGE» si Newton falla (`RuntimeError` o `LinAlgError`), si aparece un valor no finito o si $\|\omega\|$ supera 1000 veces $\|\omega_0\|$.

6. Convergencia y estimación del error

6.1 Orden de convergencia: converge_sys

`converge_sys(esquema, U0, t_i, t_f, F, J)` (`numerical_engine/convergence.py:7`) estima el orden p sin conocer la solución exacta. Para N en 10 valores geométricos entre 20 y 10 240 (`np.geomspace(20, 10_240, 10, dtype=int)`, que da 20, 40, 79, 159, ..., 5119, 10 240; ver §8.5) integra con N y con 2N pasos y mide la diferencia en el instante final:

$$E(N) = \|U_N(t_f) - U_{2N}(t_f)\| \approx C\,(1 - 2^{-p})\,N^{-p} \quad \Longrightarrow \quad \log E = \text{cte} - p \log N$$

La pendiente de la recta log E frente a log N (ajustada con `np.polyfit`, logaritmo natural; la base no afecta a la pendiente) es −p. Antes del ajuste se descartan los puntos dominados por el redondeo, $E \le 10 \cdot \varepsilon_{máq} \cdot N \cdot \max(\|U_0\|, 1)$, que aplanarían la recta (se ve en RK4 a partir de N ≈ 2000). Si quedan menos de dos puntos devuelve p = 1. El panel la ejecuta para los cinco esquemas en `mostrar_convergencia` (`GUI_actitud/panel.py:762`) con $t_f = \min(t_f, 2\text{ s})$.

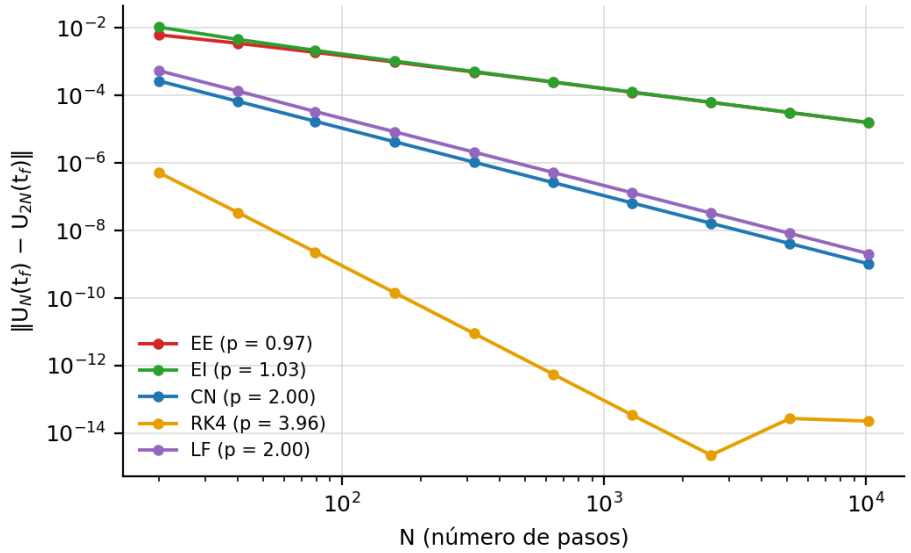


Figura 6. Convergencia con converge_sys partiendo del eje intermedio (t_f = 2 s). Órdenes estimados: EE 0.97, EI 1.03, CN 2.00, RK4 3.96, LF 2.00; coinciden con los teóricos 1, 1, 2, 4, 2.

6.2 Extrapolación de Richardson: richardson_error

`richardson_error(esquema, U0, N_t, t0, tf, F, J)` (`numerical_engine/convergence.py:26`) integra con N, 2N y 4N pasos, compara las tres soluciones en los instantes comunes y obtiene a la vez el orden y una estimación del error de la malla gruesa:

$$p \approx \log_2 \frac{\|U_N - U_{2N}\|}{\|U_{2N} - U_{4N}\|}, \qquad E_N \approx \frac{U_N - U_{2N}}{2^p - 1}$$

Si la diferencia fina es casi nula ($\leq 10^{-15}$) recurre a `converge_sys` para el orden. En el panel, `calcular_richardson` (`GUI_actitud/panel.py:722`) muestra p y $\|E(t_f)\|$ bajo el gráfico de convergencia. Como ejemplo, con N = 200 y t_f = 2 s sobre e_2: EE da p ≈ 0.98 y $\|E\|$ ≈ 8.1e-04; RK4 da p ≈ 3.99 y $\|E\|$ ≈ 3.9e-12.

A diferencia de `converge_sys`, que mide solo el instante final, las dos diferencias de `richardson_error` se calculan con la norma de toda la trayectoria. Para estimar el orden el resultado es equivalente, pero no es exactamente la misma medida.

7. La interfaz gráfica (GUI_actitud)

7.1 Distribución del panel

La ventana (`PanelActitud`, `GUI_actitud/panel.py:45`) tiene tres columnas: controles a la izquierda, un gráfico **dinámico** en el centro (se anima mientras corre la simulación) y un gráfico **estático** a la derecha (análisis que no depende del instante actual). Debajo están los deslizadores. Las vistas se eligen con dos menús desplegables (`GUI_actitud/desplegable.py`).

Vista	Tipo	Qué muestra	Se dibuja en
Movimiento del cuerpo	dinámica	Sólido 3D girando, ejes e_k, vectores L y ω, contador de volteos	`_dibujar_cuerpo` (987)
Mapa de fases	dinámica	Elipsoides de energía y momento, separatriz, equilibrios y trayectoria ω(t)	`_preparar_fase` (817), `_actualizar_graficas` (1073)
Velocidad angular ω(t)	dinámica	Las tres componentes de ω frente al tiempo	`_actualizar_graficas` (1073)
Deriva de invariantes	dinámica	$\lvert\Delta T\rvert/T_0$ y $\lvert\Delta L^2\rvert/L_0^2$ en escala log	`_actualizar_graficas` (1073)
Regiones de estabilidad	estática	Contornos ρ = 1 de EE, EI, CN y RK4, segmento [−i, i] de LF y Δt·λ del eje activo	`_actualizar_estabilidad_plot` (860)
Convergencia de integradores	estática	log E frente a log N de los 5 esquemas y resultado de Richardson	`mostrar_convergencia` (762)
Análisis espectral	estática	λ de cada eje y veredicto, 2T, L^2, cota $2I_1T \le L^2 \le 2I_3T$, max|R| del esquema	`_actualizar_analisis` (935)

7.2 Parámetros

Deslizador	Rango	Inicial	Efecto
I_1, I_2, I_3	0.05 – 3.0	los del cuerpo	Momentos principales; están encadenados para mantener $I_1 \le I_2 \le I_3$
Ω	0.5 – 6.0 rad/s	2.5	Velocidad de giro alrededor del eje elegido
ε	0 – 0.2	0.01	Perturbación: $\omega_0 = \varepsilon\Omega$ (1,1,1) con la componente del eje sustituida por Ω
Δt	0.005 – 0.05 s	0.01	Paso de integración; solo redibuja la estabilidad hasta reiniciar
t_f	5 – 60 s	30	Duración de la simulación ($N = t_f/\Delta t$)
vel.	×0.25 – ×4	×1	Velocidad de reproducción (no cambia la física)

Los deslizadores se crean en `_crear_controles` (`GUI_actitud/panel.py:399`). Al cambiar de cuerpo, `_cargar_inercias` (`GUI_actitud/panel.py:580`) carga sus momentos escalados para que $I_3 = 2$.

7.3 Bucle de simulación

```
timer (cada 1 ms) ─► _tick()                                        panel.py:705
    │  acumula tiempo real × vel y decide cuántos pasos dar (máx. 500)
    ├─► _avanzar()  (una vez por paso)                              panel.py:676
    │      ├─ paso_Cauchy(esquema, U[n], U[n-1], Δt, t, F, J)   (RK4 si LF y n = 0)
    │      ├─ comprobar divergencia
    │      ├─ R = paso_actitud(R, U[n], U[n+1], Δt)
    │      └─ contar volteos
    └─► _dibujar()  redibujo con blitting (solo lo animado)

reiniciar()                                                         panel.py:639
    lee sliders, construye ω0, F, J, invariantes y reinicia vistas
```

7.4 Catálogo de cuerpos y tensor de inercia

`GUI_actitud/cuerpos.py` define seis sólidos (raqueta, mango en T de Dzhanibekov, satélite con paneles, libro, asteroide triaxial y cohete) como uniones de piezas homogéneas: cajas, cilindros o troncos de cono y elipsoides. Cada pieza conoce su masa y sus segundos momentos; `Cuerpo` (`GUI_actitud/cuerpos.py:207`) los combina con el teorema de Steiner y obtiene el tensor de inercia por unidad de masa:

$$C = \frac{1}{M}\sum_k m_k\left(C_k + d_k d_k^T\right), \qquad \mathbf{I} = \operatorname{tr}(C)\,\mathrm{Id} - C$$

Los cuerpos están construidos para que x, y, z sean los ejes de inercia mínima, intermedia y máxima. Si el usuario cambia I_1, I_2, I_3 a mano, `deformar` (`GUI_actitud/cuerpos.py:238`) estira el dibujo para que tenga esos momentos; si $I_3 > I_1 + I_2$ (desigualdad triangular violada) ningún sólido real puede tenerlos y el panel muestra un aviso.

8. Observaciones sobre el código

Esta sección recoge lo encontrado al revisar el código para preparar el documento. Respecto a la versión anterior están resueltas la antigua 8.1 (región de Leapfrog), la 8.3 (dependencia entre paquetes) y la 8.4 (archivos `.pyc` versionados). Ninguno de los puntos pendientes afecta a la simulación principal.

8.1 Región de estabilidad de Leapfrog: corregida

Antes, el gráfico se construía con `estabilidad("LF", z)` (entonces en `space_physics/stability.py`), que devolvía solo la raíz principal $z + \sqrt{1+z^2}$. Con esa única raíz se pintaba como estable casi todo el semiplano izquierdo, cuando la región real es solo el segmento $[-i, i]$ (por ejemplo, para $z = -0.5$ la raíz usada valía 0.618 y la otra, 1.618).

Con el método nuevo, `region_por_esquema` calcula ρ como el radio espectral de la matriz de amplificación (§5.3), que tiene en cuenta las dos raíces, y el panel dibuja el segmento $[-i, i]$ (`GUI_actitud/panel.py:885`). Comprobado sobre la malla del panel: $\rho \le 1$ solo en la columna $\mathrm{Re}\, z = 0$ con $|\mathrm{Im}\, z| \le 1$, y fuera del segmento $\rho \ge 1.46$.

Además, `estabilidad("LF", z)` (`numerical_engine/stability.py:13`) se ha corregido para devolver $\max(|z + \sqrt{1+z^2}|, |z - \sqrt{1+z^2}|)$: para $z = -0.5$ da ahora 1.618. Coincide con el ρ de `region_por_esquema` hasta $3\cdot10^{-9}$ en toda la malla, y `stability_report` da exactamente los mismos valores que antes (tabla de §5.4).

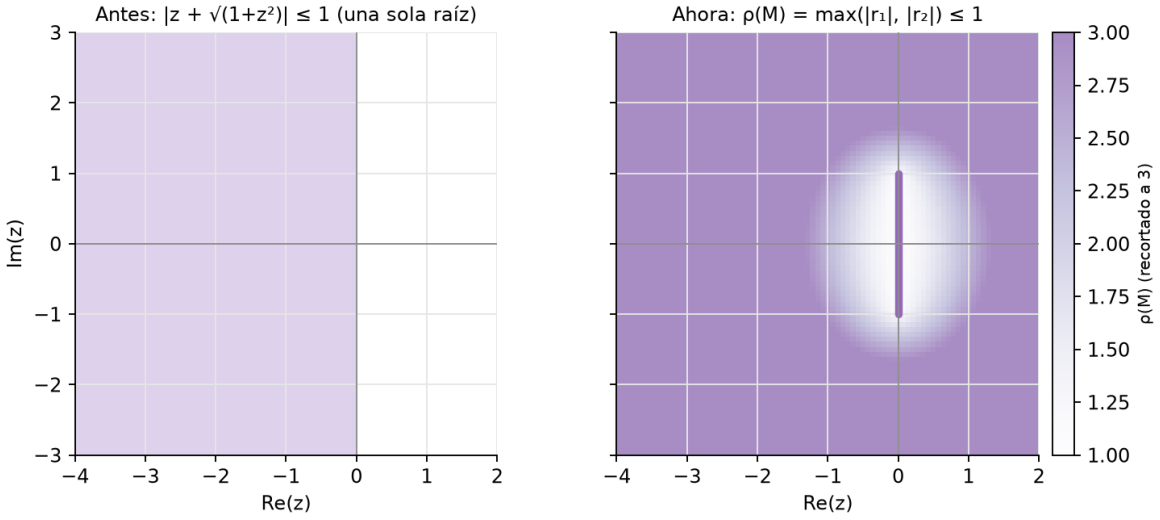


Figura 7. Izquierda: lo que dibujaba el panel antes para LF (una sola raíz). Derecha: ρ(M) calculado ahora por region_por_esquema("LF"); la región estable es el segmento [−i, i].

8.2 El jacobiano analítico no se usa en Newton

El panel pasa a EI y CN el jacobiano por diferencias finitas (`GUI_actitud/panel.py:651`), que cuesta 6 evaluaciones extra de F por iteración, aunque el jacobiano exacto ya existe en `space_physics/dynamics.py:12`. Usar `lambda W, t: jacobiano(W, I1, I2, I3)` sería más rápido y exacto. Es una mejora, no un error: con h ~ $\varepsilon^{1/3}$ la aproximación numérica es suficiente para que Newton converja.

8.3 Dependencia cruzada entre paquetes: resuelta

Las funciones R(z) estaban en `space_physics/stability.py` aunque no tienen nada de física, y `numerical_engine/stability.py` las importaba de ahí, de modo que el motor numérico dependía del paquete de física. Ahora `estabilidad` vive en `numerical_engine/stability.py:3`, el archivo de `space_physics` se ha eliminado y los dos paquetes son independientes (figura 3).

8.4 El ρ de Leapfrog se calcula pero no se usa

`region_por_esquema("LF")` calcula ρ en las 85×85 celdas, pero el panel no usa ese resultado: dibuja el segmento fijo [−i, i] (`GUI_actitud/panel.py:885`). Se podría saltar ese cálculo o, al revés, dibujar los puntos donde ρ ≤ 1 + tolerancia para que la figura salga de los datos.

8.5 Los pasos de converge_sys no son potencias exactas de 2

`np.geomspace(20, 10_240, num=10, dtype=int)` (`numerical_engine/convergence.py:8`) calcula los puntos en coma flotante y `dtype=int` **trunca**: el resultado es 20, 40, 79, 159, 319, 639, 1279, 2559, 5119, 10 240 en lugar de 20, 40, 80, …, 5120. No afecta al orden estimado (el ajuste usa los N reales), pero N ya no se duplica exactamente entre puntos. Para obtener los valores exactos basta con:

```
pasos = 20 * 2**np.arange(10)
```

9. Resumen

Idea	Dónde se ve en el programa
Los ejes de inercia mínima y máxima son estables; el intermedio no.	Vista *Movimiento del cuerpo* eligiendo cada eje: solo e_2 acumula volteos.
La causa es el signo de los autovalores del jacobiano en cada equilibrio.	Vista *Análisis espectral*: λ imaginarios (centro) frente a λ reales (silla).
T y L^2 se conservan; ω recorre la intersección de dos elipsoides.	Vista *Mapa de fases* (elipsoides, separatriz) y *Deriva de invariantes*.
EE amplifica, EI disipa, CN y LF conservan, RK4 casi conserva.	Vista *Regiones de estabilidad* (calculadas aplicando cada esquema a la ecuación test) + *Deriva de invariantes* cambiando de integrador.
Los órdenes son 1, 1, 2, 4 y 2.	Vista *Convergencia de integradores* (ajuste log-log y Richardson).